Sorting Algorithms

Bit by Bit Coding - Term 2 | Week 6

Key Concepts

- Bubble sort: swap adjacent out-of-order pairs until sorted.
- Selection sort: find the smallest, put it next.
- Insertion sort: build the sorted part one item at a time.
- sorted() returns a new sorted list; .sort() sorts in place.
- Simple sorts are O(n^2); Python's sort is O(n log n).
- Stability: a stable sort keeps the order of equal items.

Syntax Reference

Bubble Sort

```
def bubble_sort(arr):
    n = len(arr)
    for i in range(n):
        for j in range(0, n - i - 1):
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
    return arr
```

Selection Sort

```
def selection_sort(arr):
    for i in range(len(arr)):
        min_i = i
        for j in range(i + 1, len(arr)):
            if arr[j] < arr[min_i]:
                min_i = j
        arr[i], arr[min_i] = arr[min_i], arr[i]
    return arr
```

Insertion Sort & Built-ins

```
def insertion_sort(arr):
    for i in range(1, len(arr)):
        key = arr[i]
        j = i - 1
        while j >= 0 and arr[j] > key:
            arr[j + 1] = arr[j]
            j -= 1
        arr[j + 1] = key
    return arr

# Python built-ins
new_list = sorted([3, 1, 2])   # returns [1,2,3]
nums = [3, 1, 2]
nums.sort()                    # sorts in place
```

Common Patterns

- Swap two values without a temp: a, b = b, a.
- Use sorted() for a new list; .sort() to modify the original.

- Custom order with key=: sorted(words, key=len).

Tips & Common Mistakes

- .sort() returns None - don't do x = lst.sort().
- Bubble/selection/insertion sorts are simple but slow (O(n^2)).
- Python's sort is stable and very fast - prefer it in real code.
- Items must be comparable (can't sort mixed types like 3 and "a").